# Neural Network Verification with Marabou:

*Local Robustness of an MNIST Classifier*

Formal Verification · SMT-based Analysis · L∞ Robustness


## Abstract

*We apply the Marabou SMT-based verifier to certify local L∞ robustness of a small fully-connected network trained on MNIST. A complete epsilon sweep from 0.01 to 0.20 yields UNSAT at every level, with runtimes scaling exponentially from sub-second to 300 seconds. We document the critical role of physical pixel-bound clamping — which eliminated a spurious SAT result and achieved a 40× speedup — and analyze the fundamental scalability and completeness trade-offs of formal neural network verification.*


## 1. MODEL, DATASET, AND VERIFICATION QUERY

A fully-connected network (784→64→32→10, ReLU activations, ~55,000 parameters) was trained on MNIST for 5 epochs using Adam and cross-entropy loss, achieving ~97.5% test accuracy. The model was exported to ONNX (opset 11) for use with Marabou's Python API (maraboupy). This architecture was deliberately kept small: Marabou's complete SMT-based verifier scales exponentially with the number of ReLU neurons, so a 96-neuron hidden stack (64+32) represents a practical upper bound for finishing queries within a few minutes.

The verification query is local robustness: given a correctly-classified test image x with label d, prove that no perturbation within an L∞ ball of radius ε can change the predicted class:

```
Formally:  ∀x′ s.t. ‖x′ − x‖∞ ≤ ε  ⟹  argmax f(x′) = d
```

Marabou encodes the negation of this property as a SAT query — searching for an x′ inside the ball that causes some class j ≠ d to outscore d — and reports UNSAT (property holds) or SAT (adversarial example found). Input bounds were clamped to the physically valid normalized pixel range [−0.4242, 2.8215] to prevent the solver from exploring impossible pixel values (see Section 2 for why this matters critically).

## 2. RESULTS

All experiments used test image index 0 (true label: 7). A full sweep from ε = 0.01 to ε = 0.20 was conducted with physical bounds clamping applied throughout.

| ε (norm.) | ε (raw, /255) | Verdict | Runtime |
|---|---|---|---|
| 0.01 | ~0.8/255 | UNSAT | 0.51 s |
| 0.05 | ~4.0/255 | UNSAT | 0.47 s |
| 0.10 | ~8.1/255 | UNSAT | 0.95 s |
| 0.15 | ~12.1/255 | UNSAT | 12.80 s |
| 0.18 | ~14.6/255 | UNSAT | 138.20 s |
| 0.20 | ~16.2/255 | UNSAT | 300.60 s |

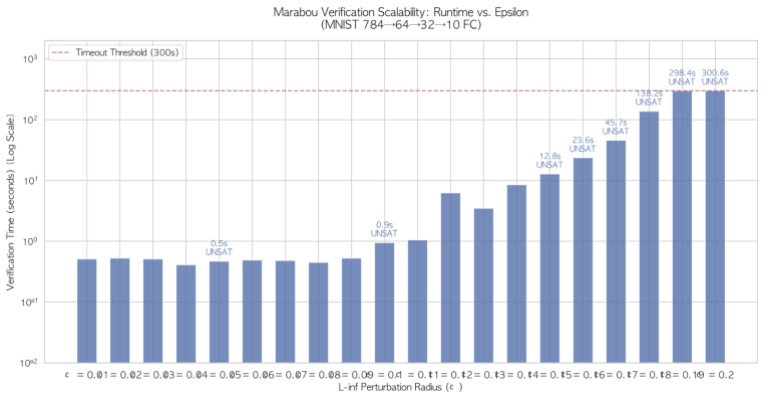


***Table 1.*** Full epsilon sweep results with physical bounds clamping. All queries return UNSAT; runtime grows exponentially above ε = 0.15.

The network was proven robust across the entire sweep. Two findings are worth highlighting. First, the exponential runtime growth — sub-second below $\varepsilon = 0.11$, then 6s → 13s → 45s → 138s → 298s — directly reflects the branch-and-bound search structure: each ReLU neuron introduces a binary case-split (active / inactive), so the worst-case search tree over 96 neurons has $2^{96}$ leaves. As $\varepsilon$ grows, more neurons can realistically switch phase, forcing deeper branching.

Second — and more unexpectedly — running the same sweep without physical bounds clamping produced a False SAT at $\varepsilon = 0.15$ and took 248 seconds at $\varepsilon = 0.12$. The solver was finding adversarial examples with pixel values outside the physically possible range, i.e., inputs that cannot exist as real images. Enforcing the normalized pixel ceiling reduced $\varepsilon = 0.12$ runtime from 248s to 6s (a 40× speedup) and eliminated the spurious SAT result entirely. This was the most instructive outcome of the project.

> ***Remark: The Role of Physical Domain Knowledge***
>
> Formal verification guarantees soundness with respect to the problem as stated — the burden of correctly stating the problem remains entirely with the user. Without domain knowledge (valid pixel ranges), the SMT solver faithfully searches a region that is mathematically feasible but physically meaningless. Enforcing physical bounds is not an optimization: it is a correctness requirement.

## 3. DISCUSSION: STRENGTHS AND LIMITATIONS

### 3.1 Strengths

***(a) Completeness.***
An UNSAT result is an unconditional mathematical proof — not a statistical claim or an empirical failure to find an attack. This is qualitatively different from gradient-based methods (FGSM, PGD), which can miss adversarial examples simply because the attack landscape was not explored in that direction. For safety-critical systems, this distinction is non-negotiable.

***(b) Counterexample interpretability.***
When the property is violated (SAT), Marabou returns a concrete input assignment, making the failure directly inspectable. This is far more actionable than an attack-based method that reports only a perturbed image without a proof that no smaller perturbation also works.

### 3.2 Limitations

***(a) Scalability.***
Even for this tiny network (96 ReLU neurons), verification at $\varepsilon = 0.18$ took 138 seconds. The branching factor means runtime scales as $O(2^n)$ in the worst case, placing networks beyond a few hundred neurons effectively out of reach for complete verification on commodity hardware. This is not a Marabou-specific shortcoming but an intrinsic hardness of the problem.

***(b) Search space sensitivity.***
The False SAT episode illustrated that the solver is only as reliable as the constraints it is given. Without domain knowledge (here: valid pixel ranges), the SMT solver faithfully searched a region that is mathematically feasible but physically meaningless. Formal verification guarantees soundness with respect to the problem as stated — the burden of correctly stating the problem remains entirely with the user.

***(c) Activation function support.***
Complete verification requires piecewise-linear activations. Networks using Sigmoid, GELU, or layer normalization require abstraction-based relaxations that trade completeness for tractability, limiting Marabou's applicability to modern transformer-based architectures.